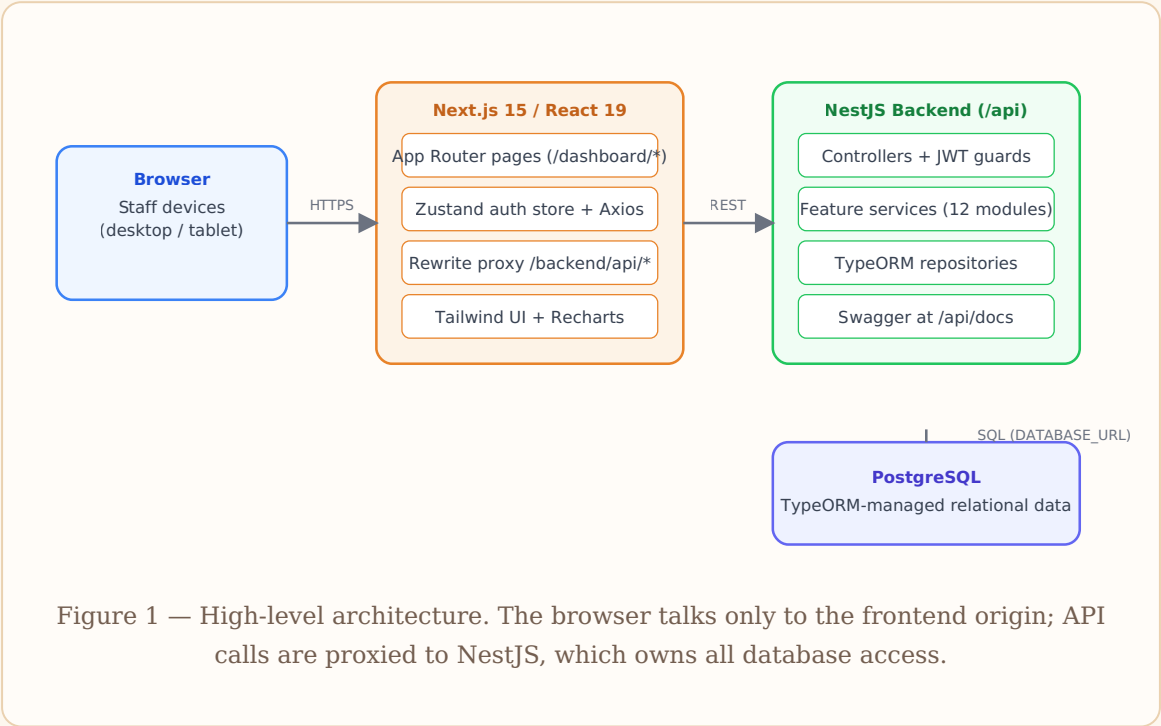


Software Design Document (SDD)

System: Abajifar — CARAVAN Lounge Restaurant Management System
Document: SDD v1.1 · **Updated:** August 20, 2026

1. System Architecture

The system is a three-tier logical application and modular monolith. The registered Abajifar web artifact runs `artifacts/nextjs-app` (frontend). `artifacts/nestjs-server` is the authoritative restaurant backend. `artifacts/api-server` is a separate minimal artifact and must not be confused with the restaurant API.



2. Frontend Structure

Path	Responsibility
------	----------------

<code>src/app/login</code>	Login page (photo hero, credential form).
<code>src/app/dashboard/page.tsx</code>	Role-aware landing: renders ManagerDashboard for admin/owner/manager; redirects waiter/coordinator/cashier to Orders.
<code>src/app/dashboard/*</code>	Feature pages: orders (POS), order-board, kitchen, tables, menu, inventory, item-requests, requests, staff, branches, reports.
<code>src/components</code>	Shared UI: sidebar (role-filtered nav), manager-dashboard, stat cards, etc.
<code>src/lib/api.ts</code>	Browser API facade; attaches JWT and calls the same-origin <code>/backend/api</code> boundary.
<code>src/store/auth.ts</code>	Zustand store: current user, token, login/logout.

Data freshness is achieved with a uniform pattern: each operational page runs `fetchData()` on mount and every 10 s via `setInterval`, without toggling loading state (no flicker).

3. Backend Structure

Module	Endpoints (prefix <code>/api</code>)	Notes
Auth	POST <code>/auth/login</code> , GET <code>/auth/me</code>	bcrypt verify → JWT (Passport strategy)
Users	<code>/users</code> , <code>/users/staff-list</code> , <code>/users/waiters</code> , <code>/users/chefs</code>	Role management
Restaurants / Branches	<code>/restaurants</code> , <code>/branches</code>	Company hierarchy
Menus	<code>/menus</code> (categories + items)	Availability flags feed POS
Tables	<code>/tables</code>	Status transitions
Orders	<code>/orders</code> (+ <code>/stats</code> , <code>/alerts</code> , item edits, status)	Lifecycle enforcement
Kitchen	<code>/kitchen</code> (board, accept/preparing/ready)	Chef queues

Payments	/payments (+ report, shifts)	Marks orders paid, frees tables
Inventory	/inventory (items, low-stock, suppliers, purchase orders, requests, adjustments)	Store operations
Notifications	/notifications	Role/user targeted
Seed	POST /seed	Demo data (Ethiopian menu, staff)

4. Database Design (ER Overview)

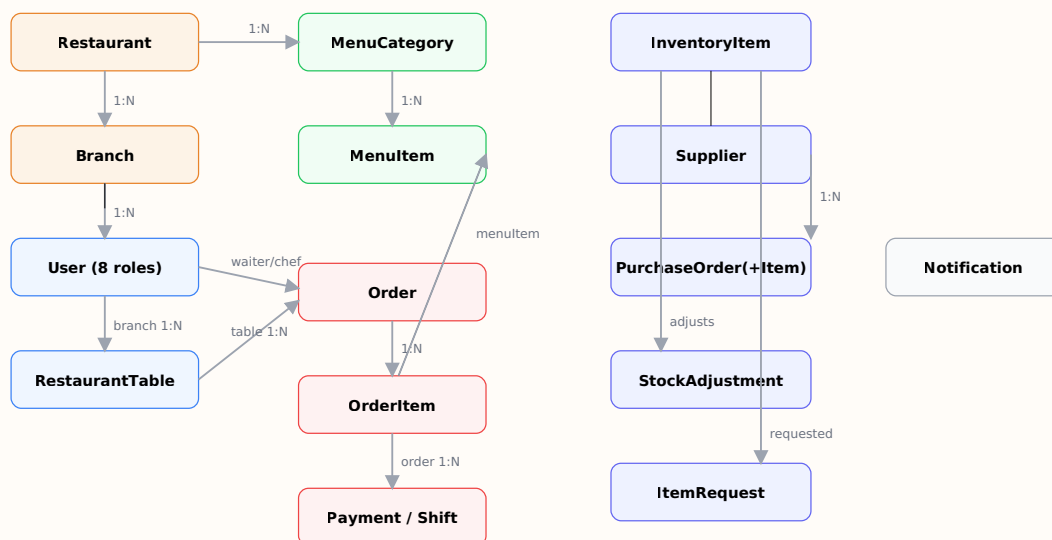


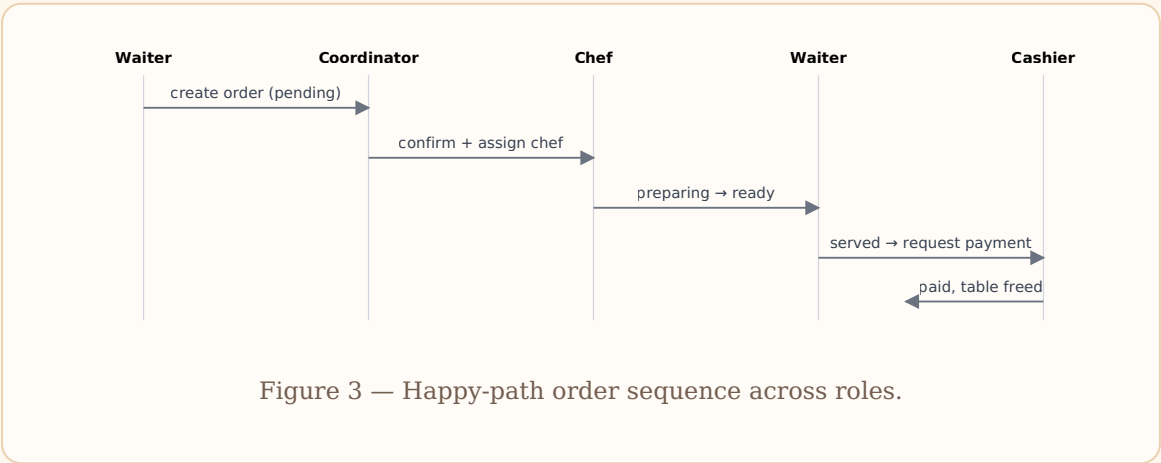
Figure 2 — Entity-relationship overview (key relations only). All entities carry auto-increment ids and timestamps where relevant.

Key columns per entity are listed in the SRS §2.3 environment and in the backend entity files (`artifacts/nestjs-server/src/**/*.entity.ts`), the authoritative source. Notable choices:

- `Order.status` enum drives the entire workflow; `OrderItem` cascades on order delete.
- `User.password` uses `select:false` so hashes never leak into queries by default.

- `ItemRequest` keeps requester / approver / issuer user references for auditability.

5. Order Lifecycle (Sequence)



6. Cross-Cutting Design Decisions

Concern	Decision
Auth	Stateless JWT (Passport). Token in the Zustand store; Axios interceptor adds the header.
API routing	Backend global prefix <code>/api</code> ; frontend proxies <code>/backend/api/*</code> via Next.js rewrites so the browser stays same-origin.
Realtime	10-second polling everywhere (simple, proxy-friendly, no WebSocket infra).
Schema	TypeORM <code>synchronize:true</code> for development speed; switch to migrations for production (see Handoff doc).
Exports	Client-side generation: <code>xlsx</code> for Excel, <code>jspdf</code> + <code>jspdf-autotable</code> for PDF, loaded dynamically.
Styling	Tailwind CSS with shared utility classes (<code>.card</code> , <code>.input</code> , <code>.table-cell</code>) in <code>globals.css</code> ; brand color <code>#E8832A</code> .
Profitability	Food cost estimated from stock deduction/waste movements × item unit cost (documented in User Manual).

7. Boundary and Readiness Notes

- The frontend owns presentation, navigation, forms, loading/error states, and API communication. It must never receive database credentials or own authoritative business permissions.
- The NestJS backend owns authentication, authorization, role/branch scope, validation, transactions, business rules, and all database access.
- PostgreSQL owns persistence and relational integrity. TypeORM entities are currently authoritative, but production migration evidence is not yet complete.
- The browser contract remains `/backend/api/*`; changing it requires coordinated frontend, rewrite, backend, deployment, and documentation updates.
- Current production blockers include schema synchronization, a fallback JWT secret, startup account mutation, wildcard CORS, incomplete DTO validation, and incomplete automated tests.

Detailed routing, environment, API, database, testing, and readiness rules are maintained in documents 07-12.